

Parrot: 以语义变量高效服务 LLM 应用

Chaofan Lin^{1*}, Zhenhua Han², Chengruidong Zhang², Yuqing Yang²

Fan Yang², Chen Chen^{1*}, Lili Qiu²

¹Shanghai Jiao Tong University, ²Microsoft Research

摘要

大语言模型 (LLM) 的兴起催生了基于 LLM 的应用, 也称人工智能智能体或副驾驶。这是一种结合 LLM 与传统软件优势的新软件范式。不同租户的多样化 LLM 应用, 可以设计由多次 LLM 请求组成的复杂工作流以完成一个任务。然而, 它们只能使用当前公共 LLM 服务过于简化的请求级 API, 丢失了关键的应用级信息。公共服务只能盲目优化单个 LLM 请求, 使应用的端到端性能未能达到最佳。

本文提出 Parrot, 一种专注 LLM 应用端到端体验的服务系统。Parrot 提出语义变量这一统一抽象, 向公共 LLM 服务暴露应用级知识。语义变量标记请求提示词中的输入或输出变量; 连接多个 LLM 请求时, 它还构成数据传递管道, 为编写 LLM 应用提供自然的方式。将语义变量暴露给公共服务后, 服务便能通过传统数据流分析, 识别多个请求之间的关联。这些关联为优化 LLM 应用端到端性能打开了新的空间。广泛的评估表明, 对流行且实用的 LLM 应用场景, Parrot 可获得最高一个数量级的性能改善。

1 引言

大语言模型 (LLM) 展现了出色的语言理解能力 [7, 41], 推动应用开发范式发生变化。在新范式中, 一个或多个应用实体, 称为人工智能智能体或副驾驶, 通过称为“提示词”的自然语言与 LLM 通信, 协同完成任务。例如, Microsoft Teams 或 Google Meet 等会议应用可通过 LLM 总结讨论 [33]; Google 和 Bing 等搜索引擎可通过 LLM 获得对话能力 [14, 34]。人们认为, 基于 LLM 的应用将在不久的将来成为主流应用 [13]。

完成一个任务时, LLM 应用通常需要多轮对话。这些对话由多次 LLM API 调用实现, 呈现复杂的工作流模式。图 1 展示了几种流行的模式。例如, 会议摘要应用 [8, 33] 常把长文档拆成较短的片段, 每片满足 LLM 对话长度限制, 再通过 Map-Reduce 或链式摘要模式, 把各片摘要合成最终摘要。Bing Copilot [34] 等对话应

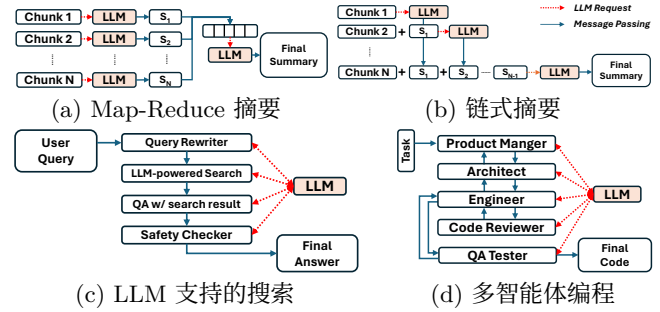


图 1: 流行 LLM 应用的工作流。最终结果需要多次 LLM 请求。Chunk 为文档片段, Final Summary 为最终摘要, LLM Request 为 LLM 请求, Message Passing 为消息传递; User Query 为用户查询, Query Rewriter 为查询改写器, LLM-powered Search 为 LLM 搜索, QA w/ search result 为基于搜索结果的问答, Safety Checker 为安全检查器, Final Answer 为最终回答; Task 为任务, Product Manager / Architect / Engineer / QA Tester / Code Reviewer 分别为产品经理 / 架构师 / 工程师 / 测试员 / 代码审查员, Final Code 为最终代码。各摘要中间结果保留原符号。

用会多次调用 LLM API, 根据用户查询生成回答。多个智能体也可协作完成任务, 每个智能体代表不同角色, 由不同 LLM 调用实现 [22, 47, 54]。

公共 LLM 服务提供商面对不同租户和应用, 各自具有不同工作流和性能偏好。然而, 现有服务 API 仍以请求为中心。服务只能观察到大量孤立请求, 不知道哪些请求属于同一应用、不同请求如何连接、是否存在相似内容等应用级信息。这些信息丢失, 使公共服务盲目优化单个请求, 导致应用的端到端性能次优。我们观察到, 利用应用级信息, 尤其是多个 LLM 请求之间的关联, 能显著改善应用的端到端体验。

第一, 连续的多个 LLM 请求可能相互依赖: 前一请求的结果可直接作为下一请求的输入。因此, 最好把这些请求放在同一位置, 并在服务侧连续执行。然而, 服务不知道依赖关系, 这些请求只能在应用客户端与公共 LLM 服务之间交互执行。客户端通常位于互联网另一端, 只有收到前一请求结果后才能提交下一请求。这会

*部分工作完成于 Chaofan Lin 在 Microsoft Research 实习及 Chen Chen 博士访问该机构期间。

引入额外网络时延，也丧失联合调度连续请求的机会，造成连续请求的额外开销 (§3)。

第二，即使同一应用内部，LLM 请求也可能具有不同调度偏好。例如，在图 1a 中，为缩短端到端时延，应更积极地把多个 Map 任务组成批次，提高吞吐量；Reduce 任务数量较少，则应优化其时延。然而，公共 LLM 服务无法区分这两类任务，只能盲目优化各请求时延，这未必有利于端到端体验。

第三，LLM 请求之间存在大量共性。Bing Copilot [32] 和 GPTs [42] 等流行应用使用较长的系统提示词，包括任务定义、示例和安全规则，以约束应用行为。这些系统提示词通常固定，所有用户共用。然而，现有公共服务把请求独立处理，每次请求重复携带公共前缀，浪费存储、计算和内存带宽。我们对生产环境 LLM 搜索引擎的分析发现，不同用户请求中的词元有超过 94% 是重复的。

虽然 Orca、vLLM、SGLang 等新兴引擎级技术 [25, 56, 63] 已针对上述三类情况进行优化，它们都依赖某些应用级知识，而这些知识在当今公共 LLM 服务中丢失了。综上，由于不理解 LLM 请求之间的关联，现有服务无法利用这三类机会，造成较高端到端时延和较低吞吐量。

基于这些事实与观察，我们提出 Parrot，将 LLM 应用作为一等对象。Parrot 使用简单的语义变量抽象，保留大多数应用级信息，在系统复杂度与新增优化信息之间取得平衡。语义变量是提示词中具有特定语义用途的文本区域，例如任务指令、少样本示例、输入或输出。语义变量也可以作为连接多个 LLM 请求的数据传递管道。它自然地与服务暴露提示词结构和请求关联。通过在运行时检查语义变量，Parrot 可执行传统数据流分析，及时推导 LLM 请求之间的数据依赖。

分析应用级信息后，Parrot 的统一抽象自然支持联合优化，有助于取得更好的整体效果。同一个由语义变量组成的数据管道，可以同时支持多项优化，包括隐藏管道时延、通过目标推导改善调度，以及通过共性分析去除重复内容。Parrot 的调度也在统一抽象下共同考虑不同优化机会。我们在流行 LLM 应用上进行广泛评测，涵盖生产应用与开源项目；结果显示，与当时先进方案相比，Parrot 最高实现 11.7× 加速，或 12× 吞吐量。

2 背景

LLM 服务。 大多数 LLM 服务通过文本补全 API，提供条件文本生成服务：

$Completion(prompt : str) \rightarrow generated_text : str.$

应用客户端提供文本提示词，LLM 服务返回生成的文本。API 背后，服务提供商运行一个或多个 LLM 推理引擎集群。请求调度器把队列中的请求派发到某个引擎，再由引擎使用一组 GPU 执行推理。上式中 Completion 为补全接口，prompt 为提示词，generated_text 为生成文本，str 为字符串类型。

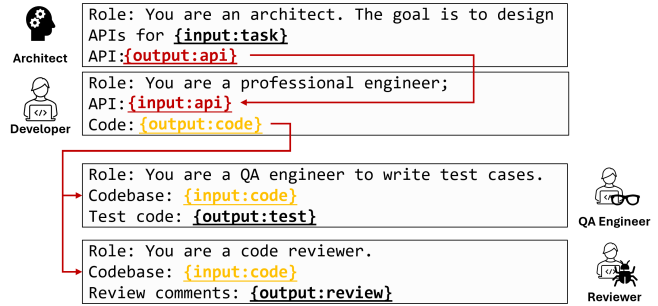


图 2: 多智能体应用中连续 LLM 请求之间的通信。Architect 为架构师：角色是架构师，目标是为输入任务设计 API，输出 api；Developer 为开发者：角色是专业工程师，以 api 为输入，输出 code；QA Engineer 为测试工程师：角色是编写测试用例的 QA 工程师，以 code 为代码库输入，输出 test 测试代码；Reviewer 为审查员：角色是代码审查员，以 code 为代码库输入，输出 review 审查意见。input / output 标记输入 / 输出占位符。

基于 LLM 的应用。 图 1 展示了应用使用 LLM 的典型 workflow。LLM 的上下文窗口有限，例如当时 GPT-3.5-Turbo 为 4096 [40]；长文档分析因此通过 Map-Reduce 风格 (图 1a) 或链式 (图 1b) 工作流产生最终结果：将长文本分片，多次请求分别生成片段结果 (Map 任务)，再整体合并 (Reduce 任务)，或逐步合并 (链式)，得到最终结果。图 1c 的对话搜索引擎可能连续调用 LLM，识别查询意图、补充查询信息、检索相关数据、进行安全检查，最后生成回答。图 1d 与图 2 的多智能体应用也是使用多次 LLM 请求的工作流，每次请求承担指定角色。例如，AutoGen [54] 和 MetaGPT [22] 使用产品经理、架构师、工程师、测试员等角色，围绕同一软件项目交流与协作；每个角色由一个或多个 LLM 请求支持，生成相应回复。

3 LLM 应用服务面临的问题

文本补全 API 虽然为构建 LLM 应用提供了灵活方式，即使公共服务丢失应用级信息，从而引发以下挑战。

连续请求的过多开销。 如图 1 所示，LLM 应用常要多次调用模型才能完成任务。现有公共服务以请求为中心，逐个独立生成响应；开发者必须在客户端解析前一请求的输出，拼接后续提示词。图 3a 展示了对生产环境中某流行 LLM 应用的实测时延构成。应用使用链式工作流，提示词长度为 150 到 4000 个词元，输出长度约 50 个词元。我们发现，LLM API 调用时延中有很大一部分发生在引擎之外，平均占 30 ~ 50%，最坏超过 70%。提示词越长，额外开销越大；高时延有时会造成 API 超时和重新提交。

这一开销源于服务与客户端之间的频繁交互。图 3b 展示一个简单两步应用的开销，例如对两个文本片段进行链式摘要。现有服务不知道请求之间的依赖，即前一

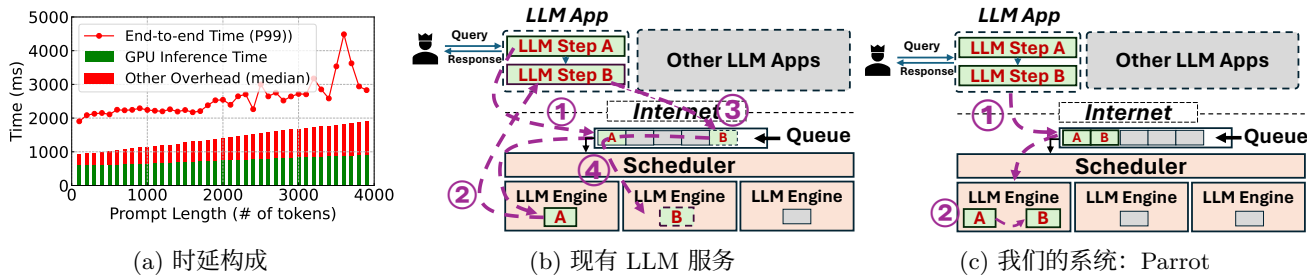


图 3: 现有 LLM 服务的端到端时延构成。应用与服务频繁交互，带来网络 and 排队开销；Parrot 消除了这部分开销。左图 Prompt Length 为提示词长度（词元数），Time 为时间（毫秒），End-to-end Time (P99) 为端到端时间第 99 百分位数，GPU Inference Time 为 GPU 推理时间，Other Overhead (median) 为其他开销中位数。右侧流程图中，LLM App 为应用，Other LLM Apps 为其他应用，Step A / B 为步骤 A / B，Internet 为互联网，Queue 为队列，Scheduler 为调度器，LLM Engine 为引擎，Query / Response 为查询 / 响应；圈中编号表示交互与排队过程。

请求输出可能直接用作下一请求输入。对连续依赖请求，客户端必须先收到第一请求的响应 (②)，才能提交下一请求 (③)。客户端与服务通常处于不同数据中心，这会无谓地增加网络时延。此外，连续请求之间可能插入其他应用的请求，下一请求还需额外排队 (④)。

表 1 评测了四种流行 LLM 应用，前两种来自生产环境，后两种是开源项目。原文指出，这些应用完成一个任务需要数十次 LLM 调用，导致较高的用户感知时延。§8.2 的实验表明，独立处理各请求会使端到端时延恶化超过 2×。如果服务能成组处理连续请求，就可以消除这些开销。¹

Parrot 采用这一方法。如图 3c 所示，同一应用的两个步骤一起调度，步骤 A 的输出直接送入步骤 B，绕过额外网络与排队开销。

调度目标不一致。 由于丢失工作流与应用性能目标等信息，现有公共服务只能对所有请求采取统一处理方式，例如优化每个请求的时延 [44]。然而，LLM 应用更关心端到端体验。优化目标的不一致可能损害整体性能。以图 1a 的文档摘要为例，系统应最小化用户获得最终摘要的总时间，而非各请求时延；针对单个请求优化，并不能使端到端时延最优。

如图 4 所示，现有服务为控制单请求时延，必须限制每个引擎的并发请求数。然而，LLM 推理存在时延与吞吐量权衡：增大批量大小可使吞吐量最高提高至 8.2×，同时使时延增加 95% [9]。若已知应用级目标是端到端时延，便能推导出更合适的策略：在 Map 阶段使用更大批次，最大化吞吐量；在 Reduce 阶段最小化请求时

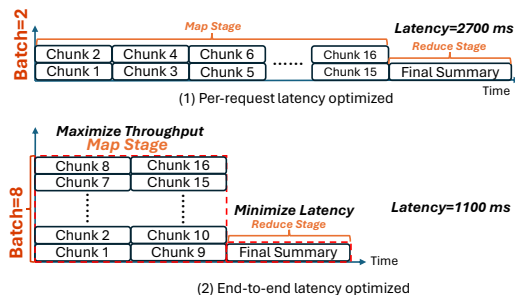


图 4: Map-Reduce 文档摘要中，以请求为中心与以应用为中心的调度对比。Chunk 为文档片段，Final Summary 为最终摘要，Map / Reduce Stage 为映射 / 归约阶段，Batch 为批量大小，Time 为时间，Latency 为时延；(1) 优化单请求时延，(2) 优化端到端时延。Maximize Throughput 为最大化吞吐量，Minimize Latency 为最小化时延。图例给出总时延分别为 2700 和 1100 毫秒。

延。这使端到端执行获得 2.4× 加速，也说明可以在不损害应用端到端时延的前提下提高集群吞吐量。面对需求增长与硬件资源有限的矛盾，这一观察很重要。它强调必须从应用视角调度 LLM 请求，也带来管理具有不同性能目标的请求这一挑战。

重复计算。 当前大多数 LLM 应用的请求提示词有很高的重复度。例如，Bing Chat [32] 已处理超过十亿条对话提示词，它们共享定义系统功能的相同系统提示词。OpenAI 的 GPTs [42] 允许用户针对特定目的定制 ChatGPT，不同用户也共享同一提示词模板。提示词共性很重要，因为它规定应用功能与限制。图 5 [52] 中，提示词包含角色定义、用于提高行为准确性的示例及用户查询详情。用户输入是动态的，但任务角色固定，同类任务又可共享同一组少样本示例，因此示例基本固定。这解释了为什么不同用户请求可重复使用超过 94% 的前缀词元 (表 1)。多智能体应用也存在这种共性。例如，MetaGPT [22] 与 AutoGen [54] 在多轮请求中反复加入对话历史，重复度分别达到 72% 和 99%。这些重复区域多次计算并过度消耗 GPU 内存带宽。已有工作在引擎层避免共享提示词重复占用显存 [25]，但公共服务难

LLM 应用	调用次数	词元数	重复比例 (%) *
长文档分析	2 ~ 40	3.5k ~ 80k	3%
对话搜索	2 ~ 10	5k	94%
MetaGPT [22]	14	17k	72%
AutoGen [54]	17	57k	99%

* 某段文本若出现在至少两次 LLM 请求中，便计为重复。

表 1: LLM 应用的调用统计。

¹ 译注：表中也包含仅需 2 次调用的任务，因此“都需要数十次”是原文的概括表述，具体调用范围以表格为准。

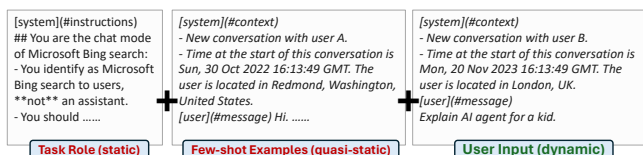


图 5: Bing Copilot 提示词结构展示了不同用户查询复用的长提示词。Task Role (static) 为固定任务角色, Few-shot Examples (quasi-static) 为基本固定的少样本示例, User Input (dynamic) 为动态用户输入。左栏指令意为: “你是 Microsoft Bing 搜索的对话模式; 向用户表明自己是 Microsoft Bing 搜索, 而非助手; 你应当……” 中栏示例意为: “与用户 A 开始新对话; 对话开始于 2022 年 10 月 30 日星期日 16:13:49 GMT; 用户位于美国华盛顿州雷德蒙德; 用户: 你好……” 右栏意为: “与用户 B 开始新对话; 开始于 2023 年 11 月 20 日星期一 16:13:49 GMT; 用户位于英国伦敦; 用户: 向小孩子解释人工智能智能体。” system / user 分别标记系统 / 用户, instructions / context / message 分别为指令 / 上下文 / 消息。

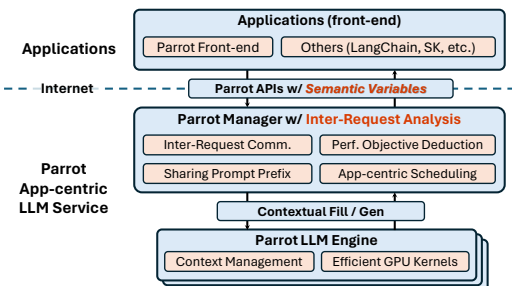


图 6: Parrot 系统概览。Applications (front-end) 为应用前端, Parrot Front-end 为 Parrot 前端, Others 为 LangChain、Semantic Kernel 等其他前端; Parrot APIs w/ Semantic Variables 为带语义变量的 API, Internet 为互联网, Parrot Manager w/ Inter-Request Analysis 为具有跨请求分析的管理器, App-centric LLM Service 为以应用为中心的 LLM 服务; Inter-Request Comm. 为跨请求通信, Perf. Objective Deduction 为性能目标推导, Sharing Prompt Prefix 为共享提示词前缀, App-centric Scheduling 为应用中心调度; Contextual Fill / Gen 为带上下文的填充 / 生成, Parrot LLM Engine 为引擎, Context Management 为上下文管理, Efficient GPU Kernels 为高效 GPU 内核。

以从大量不同应用的请求中快速识别并放到同一引擎执行那些共享提示词、可能动态生成的请求。不知道提示词结构时, 对每个请求逐词元匹配的成本很高。若集群调度器无法把共享提示词的请求分配到同一引擎, 引擎级去重优化便难以发挥作用。

4 Parrot 设计

图 6 展示 Parrot 的设计概览。Parrot 通过语义变量标注, 为编写 LLM 应用提供自然方式 (§4.1), 并兼容

```
import Parrot as P
from Parrot.PerformanceCriteria import LATENCY
```

```
@P.SemanticFunction
def WritePythonCode(task: P.SemanticVariable):
    """ You are an expert software engineer.
        Write python code of {{input:task}}.
        Code: {{output:code}}
    """
```

```
@P.SemanticFunction
def WriteTestCode(
    task: P.SemanticVariable,
    code: P.SemanticVariable):
    """ You are an experienced QA engineer.
        You write test code for {{input:task}}.
        Code: {{input:code}}.
        Your test code: {{output:test}}
    """
```

```
def WriteSnakeGame():
    task = P.SemanticVariable("a snake game")
    code = WritePythonCode(task)
    test = WriteTestCode(task, code)
    return code.get(perf=LATENCY), test.get(perf=LATENCY)
```

图 7: 示例: Parrot 中的多智能体应用。代码及占位符保留原样: WritePythonCode 为编写 Python 代码的语义函数, 其提示词意为 “你是专业软件工程师, 编写输入 task 所描述任务的 Python 代码; 代码输出到 code”; WriteTestCode 为编写测试的语义函数, 提示词意为 “你是经验丰富的测试工程师, 为输入 task 编写测试; 输入代码为 code; 测试代码输出到 test”。WriteSnakeGame 将任务设为 “贪吃蛇游戏”, 连接两次请求, 再通过 get 以 LATENCY (时延优先) 目标取回两个输出。input / output 标记输入 / 输出。

LangChain [8] 等现有编排框架。围绕这一抽象, Parrot 管理器在集群层调度请求, 推导应用级知识 (§4.2), 并优化端到端性能 (§5)。管理器将请求调度到 LLM Engine; 一个引擎由集群中的一台 GPU 服务器或一组服务器组成, 可以独立服务请求。

4.1 语义变量

Parrot 把 LLM 请求看作通过自然语言实现、由 LLM 执行的语义函数。²语义变量定义为语义函数的输入或输出变量, 在提示词中以占位符引用。图 7 给出类似 MetaGPT [22] 的简化多智能体示例: 它包含两个 SemanticFunction, 一个由软件工程师编写代码, 另一个由测试工程师编写测试代码。三个语义变量分别为 task、code 和 test, 对应任务描述、待编写代码和待编写测试。LangChain [8] 等框架也支持提示词占位符, 但在提交前会把它们替换成实际数据, 公共服务因而看不到这种结构。Parrot 用语义变量保留提示词结构, 以便服务侧开展跨请求分析。

除语义函数外, 开发者还可定义编排函数, 连接多个语义函数, 例如图 7 的 WriteSnakeGame。这些语义变量

² “语义函数” 这一术语借自 Semantic Kernel [36]。

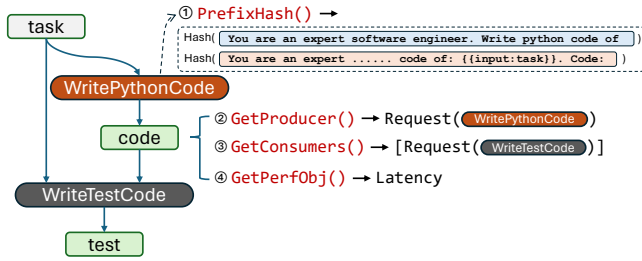


图 8: 跨请求分析原语节选。task / code / test 为任务 / 代码 / 测试变量, WritePythonCode / WriteTestCode 为编写代码 / 测试的函数; PrefixHash 为前缀哈希, Hash 为哈希运算; GetProducer 返回变量的生产者请求; GetConsumers 返回消费者请求列表; GetPerfObj 返回性能目标, 此例为 Latency (时延)。图中两段被哈希的提示词分别意为: “你是专业软件工程师, 编写……的 Python 代码”, 以及 “你是专业……编写输入 task 的代码; 代码:”。占位符及方法名保留原标识符。

在公共服务侧形成多请求的数据传递管道。对语义函数执行简单数据流分析, 就能识别请求连接。例如, 图 7 中的 code 连接 WritePythonCode 与 WriteTestCode 产生的两个请求, 显示其顺序依赖。与传统补全 API 不同, Parrot 将补全请求拆成 submit 与 get 两个操作 (§7)。调用 SemanticFunction 会触发 submit, 提交带提示词和输入语义变量的请求; 执行是异步的, 立即返回输出语义变量的 future (未来结果句柄)。应用可按需通过 get 从公共服务读取输出变量。异步设计使服务能接收所有未被原生函数阻塞的请求, 及时分析其关系。

get 支持标注性能指标, 表达应用端到端要求, 例如端到端时延或吞吐量; 还可扩展到流式输出的每词元时延、首词元时间等指标。例如, 图 7 的最终输出 code 和 test 均通过 get 标记端到端时延目标。中间变量的性能指标会从最终输出自动推导并传播 (§5.2)。传播完成后, 每个变量都带有一个指标, 作为调度器的提示 (§5.4)。

4.2 跨请求分析的原语

总体上, Parrot 主要通过语义变量推导两类应用级信息: 请求 DAG 和提示词结构。图 8 展示图 7 示例的 DAG, 以及用于跨请求分析和优化的原语。

基于 DAG 的分析。 请求或 SemanticFunction 会提前提交, 因此 Parrot 可在服务侧一次接收它们, 并即时分析关联。Parrot 在每个用户注册的会话中维护类似 DAG 的结构; 节点为请求, 或连接请求的语义变量。新请求到达时, 系统根据提示词占位符引用的语义变量添加连边, 将请求插入 DAG。Parrot 可使用传统数据流分析 [1,38], 通过 GetProducer 和 GetConsumers 得到变量生产者与消费者, 恢复请求依赖。结合请求 DAG 与最终输出变量上由 GetPerfObj 取得的性能目标, Parrot 可以推导请求级调度偏好 (§5.2)。

基于提示词结构的分析。 根据语义变量声明的结构, Parrot 能在由变量划分的多个位置提取请求前缀哈希, 原语为 PrefixHash。例如, WritePythonCode 有两个可能共享的前缀: {{input:task}} 之前的文本, 以及 {{output:code}} 之前的文本, 因此生成两个前缀哈希值。服务利用这些哈希快速识别跨请求共性, 既支持静态内容, 也支持动态生成内容; 既适用于同类应用内部, 也适用于不同应用之间 (§5.3)。

5 利用语义变量进行优化

5.1 服务相互依赖的请求

避免不必要的客户端执行, 需要知道应用级请求依赖, 而当今公共 LLM 服务丢失了这些信息。通过 §4.2 的 DAG 与原语, Parrot 使用图执行器高效处理依赖请求。执行器持续轮询, 一旦请求就绪, 即所有生产者请求均已完成, 就将其发送给相应引擎, 立即执行并尽量增加批处理机会。连续执行依赖请求时, 变量的具体值通过为相应语义变量分配的消息队列传递, 避免客户端与服务之间不必要的频繁通信。

请求中的语义变量值在传递前可能需要转换。例如, 从请求的 JSON 格式输出中提取变量值, 再送入后续请求。与 Kafka [5] 等支持消息转换的队列系统类似, Parrot 也支持在跨请求交换变量值时进行字符串转换, 并支持 LangChain [8] 的大多数输出解析方式, 覆盖主要应用场景。

5.2 性能目标推导

优化应用端到端性能, 需要知道应用级指标; 从最终要求推导请求级调度偏好, 还需要理解工作流, 即由 Parrot 原语构建的请求 DAG。

若应用标记某语义变量应优先获得高吞吐量, 则直接或间接生成该变量的全部请求, 在调度时都被标记为吞吐量优先。这通常适合批量文档分析等离线数据处理。

处理时延敏感的应用更复杂。如图 4 所示, 降低端到端时延有时需要在 Map 阶段优先保证吞吐量, 牺牲单个请求时延来缩短整个 DAG 的完成时间。Parrot 从连接到时延关键变量的请求出发, 按逆拓扑顺序分析, 如图 9 所示。直接产生时延关键输出的请求 1、2 被标为时延敏感, 其直接前驱请求 3 也如此。同一阶段的并行请求合成一个任务组, 如任务组 0 和 1。调度器应最小化整个任务组时延, 这通常意味着采用更大的批次容量, 提高词元生成吞吐量。

5.3 共享提示词前缀

请求调度到引擎后, 会创建上下文保存该请求的模型执行状态, 主要是 KV 缓存。已有工作在引擎中共享公共提示词前缀的缓存, 减少显存占用。但如 §3 所述, 公共服务面对多样化应用与请求, 难以在集群层识别共性; 大量长上下文请求逐词元比较的复杂度很高, 实际不可行。向服务暴露语义变量后, Parrot 能理解提示词结构, 以

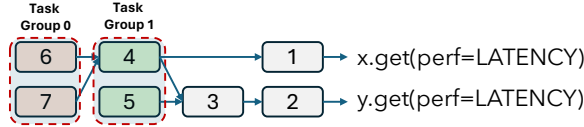


图 9: 为产生两个时延敏感语义变量的 LLM 应用推导性能目标。x.get(perf=LATENCY) 与 y.get(perf=LATENCY) 表示按时延优先目标取回两个最终输出; Task Group 0 / 1 为任务组 0 / 1, 节点数字表示请求编号。

变量为粒度更高效地自动识别共性。借助 PrefixHash, 只需检查提示词中各语义变量之后的位置所对应的哈希值。Parrot 维护键值存储, 将词元前缀的哈希映射到请求列表, 调度器便可快速在线查找共享机会, 支持同一应用或不同应用的静态与动态生成提示词。

我们进一步为共享前缀的请求设计更高效的注意力 GPU 内核。首先使用 vLLM 的分页内存管理 [25], 减少重复显存占用; 但 vLLM 内核仍会重复计算和加载共享词元。因此, 我们结合 FlashAttention [12] 与 PagedAttention [25], 设计新的解码注意力算法, 分别处理共享和非共享词元, 显著加速共享上下文的注意力计算, 细节见 §7。

5.4 以应用为中心的调度

为解决公共服务盲目优化多样化独立请求的问题, Parrot 调度利用应用级知识优化端到端性能。调度器的主要目标是在提高 GPU 集群利用率的同时, 满足不同应用的性能目标。如 §3 所述, 吞吐量优先与时延优先请求混合时存在冲突: 大批次提升吞吐量和 GPU 效率, 却损害时延, 反之亦然。基于 Transformer 的 LLM 推理很大程度上受内存带宽限制, 时延受引擎内并发词元数影响。为满足性能目标, 尤其是时延目标, 引擎需将词元数控制在阈值以下, 该阈值由时延约束最严格的请求决定。因此, Parrot 遵循两条原则: (1) 把性能要求相近的请求分组, 避免冲突; (2) 尽量增加请求间共享的机会。

算法 1 展示调度流程。系统依据抽取的 DAG, 将请求按拓扑顺序排列 (行 1)。Parrot 倾向于把同一应用的请求一起调度, 避免交错调度造成减速 (§8.2)。对于目标推导识别出的任务组, 调度器尽量整体分配 (行 4–行 5)。若发现其他排队请求或运行上下文具有公共前缀, 也尽量分配到同一引擎 (行 3、行 6–行 9), 利用上下文分叉, 减少重复计算与 GPU 内存事务。没有这些机会的请求则单独调度 (行 10–行 11)。篇幅所限, 原文省略 FindEngine 选择引擎的具体实现。简言之, 它选择满足请求调度偏好、同时负面影响最小的引擎。例如, 若把一个时延敏感请求放入可处理 64000 个吞吐量优先词元的引擎, 为满足严格时延要求, 引擎容量会显著降至 2000; 若放入已经运行时延敏感请求的引擎, 容量减损则很小。³

³译注: 伪代码的 FindSharedPrefix 查找共享前缀请求与引擎上下文, FindEngine 选择引擎; TaskGroup 为任务组, SharedReqsInQueue 为队列内共享请求, CtxInEngine 为引擎内上下文, filter

算法 1: Parrot 的请求调度。

```

输入: Q: 请求队列
1 Q.sort(); /* 拓扑顺序 */
2 对  $r \in Q$  执行
3   SharedReqsInQueue, CtxInEngine =
     FindSharedPrefix(r);
4   若  $r.TaskGroup \neq \emptyset$  则
5      $r^* = \text{FindEngine}(r.TaskGroup)$ ;
6   否则若  $SharedReqsInQueue \neq \emptyset$  则
7      $r^* = \text{FindEngine}(SharedReqsInQueue)$ ;
8   否则若  $CtxInEngine \neq \emptyset$  则
9      $r^* = \text{FindEngine}(r, \text{filter}=CtxInEngine)$ ;
10  若  $r^* = \emptyset$  则
11     $r^* = \text{FindEngine}(r)$ ;
12  Q.remove( $r^*$ );

```

6 讨论

动态应用与函数调用。 当前 Parrot 只支持不涉及动态控制流和 Python 代码等原生函数的云侧 LLM 请求编排; 这些部分仍需客户端执行。我们有意禁止把原生函数卸载到公共服务, 以减少恶意注入带来的安全风险。对于应用可信, 或具备可信函数执行区的私有服务, API 可扩展为支持条件连接和原生代码提交。这样的扩展还会产生新优化, 例如依据历史性能记录, 推测性地预先启动动态应用中高概率的分支。这展示了设计面对新应用类型的潜力; 这些扩展留待未来工作。⁴

跨请求分析的其他应用。 Parrot 打开的优化空间不限于 §5 的例子。大型服务还需考虑异常慢任务 [3]、作业失败 [58]、延迟调度 [57]、公平性 [15, 61]、饥饿 [17] 和异构集群支持 [24, 37], 其他系统已广泛研究这些问题。Parrot 从 LLM 应用视角提供了新的观察: 必须理解请求连接与共性, 才能优化应用端到端性能。结合 LLM 应用新特点, 可重新研究这些调度功能。本文聚焦机制及部分使用场景, 其他优化是有前景的未来工作。

与 LLM 编排框架结合。 LangChain [8]、Semantic Kernel [36] 和 PromptFlow [35] 等框架帮助开发者构建 LLM 应用, 核心功能是把不同 LLM 调用“粘合”起来完成复杂任务, 也称 LLM 编排。通过在这些框架的服务 API 调用中增加语义变量, 即可集成 Parrot。大多数框架已经采用带占位符的提示词模板, 并在运行时替换占位符, 其概念自然对应语义变量。然而, 框架在提交前渲染模板, 使服务丢失结构信息。为兼容 Parrot, 应把模板

为筛选约束。原伪代码把 FindEngine 的返回值赋给 r^* , 最后以 Q.remove(r^*) 移除队列项, 却未明确 r^* 是引擎、被调度请求还是请求集合, 本文保留这一记法; 调度策略不是附有最优性证明的精确优化算法。

⁴译注: 概率分支与推测性执行在此属于作者讨论的未来扩展, 本文没有实现相应概率 DAG 调度器, 也没有对这项扩展进行实验评估。

和用于填充模板的变量一起封装为 `SemanticFunction`，变量使用语义变量表示，从而向服务暴露必要信息。

7 实现

Parrot 是面向 LLM 应用的端到端服务系统，主要以 Python 实现，代码约 14000 行。前端提供语义变量和 `SemanticFunction` 抽象，并将其转换为基于 FastAPI [48] 实现的服务 API，以 LLM 请求形式提交。集中式管理器负责请求管理，包括语义变量、通信和调度。我们还基于 vLLM [25]、xFormers [26] 及自行设计的高效内核构建推理引擎，支持分页内存管理 [25] 和连续批处理 [56] 等先进功能。前端和管理器分别包含 1600 与 3200 行 Python；推理引擎包含 5400 行 Python 和 1600 行 CUDA。对于 OPT [60] 和 LLaMA [51]，我们使用 PyTorch [45] 与 Transformers [53] 实现。

API。 通过 `SemanticFunction` 或其他前端编写的应用，经不同适配器转换为统一 API 请求。Parrot 在类似 OpenAI 的 API 上扩展了语义变量。§4.1 所述 `submit` 与 `get` 两个操作的请求体如下：

```
(submit) {"prompt": str, "placeholders": [{"name": str,
→ "in_out": bool, "semantic_var_id": str,
→ "transforms": str}, ...], "session_id": str}

(get) {"semantic_var_id": str, "criteria": str,
→ "session_id": str}
```

除了静态提示词字符串，Parrot 还保留输入和输出占位符。每个占位符关联一个语义变量，用于填充输入或解析输出。如 §5.1 所述，系统支持输入前或输出后的转换，也提供设置与获取变量值的其他 API。获取某变量时，若中间步骤失败，包括引擎、通信或字符串转换失败，会返回错误消息。⁵

内核优化。 vLLM 的 GPU 内核虽然能复用显存中已缓存的公共前缀词元结果，但有时会反复将数据从全局内存加载到共享内存，拖慢注意力分数计算。我们使用 OpenAI Triton [43] 和 CUDA 开发新内核，结合 PagedAttention [25] 与 FlashAttention [11, 12] 的思想，加速共享前缀的解码注意力计算。它保留 PagedAttention 将 KV 缓存存放在不连续内存区间的方式，用每请求页表追踪缓存块状态和位置。

内核借助 FlashAttention 原理，尽量复用共享内存数据。与 PagedAttention 重复加载数据块不同，它仅把公共前缀的 KV 缓存块加载到共享内存一次，减少 L2 缓存与共享内存之间的事务。内核先使用已加载的数据块，计算公共前缀部分的中间注意力，包括注意力分数、`qk_max` 和 `exp_sum`，将它们写回高带宽内存 HBM；

随后计算前缀之外新词元的部分注意力，再与前缀的中间结果合并，得到最终注意力输出。⁶

通用引擎抽象。 Parrot 集群管理器控制多个引擎，它们可能运行不同模型、分词器和 KV 缓存布局。要支持本文优化，引擎至少需要：(1) 有状态生成，例如 `guidance` [18]；(2) 跨请求共享 KV 缓存状态。因此，我们提出通用抽象，描述引擎接入 Parrot 所需的最小能力。

```
def Fill(token_ids: List[int], context_id: int,
→ parent_context_id: int)
def Generate(sampling_configs: Dict, context_id: int,
→ parent_context_id: int)
def FreeContext(context_id: int)
```

以下三个方法既覆盖基础补全功能，也提供灵活的上下文管理接口。`Fill` 处理初始提示词词元，计算并填入相应上下文的 KV 缓存。`Generate` 按温度等采样配置执行解码，每轮生成一个词元，直到达到长度限制、用户指定终止字符或 EOS（序列结束）词元。引擎调度器利用连续批处理 [56]，按迭代调度与合批 `Fill` 和 `Generate`。分别设置 `context_id` 与 `parent_context_id`，还可创建或分叉上下文。`FreeContext` 显式释放上下文，即释放其显存 KV 缓存。把 `Fill` 与 `Generate` 分开，不仅自然对应语义变量的用法，即常量文本和输入值由 `Fill` 处理、输出值由 `Generate` 生成，还把请求级依赖细化，创造更多并行机会 [2, 21, 46, 64]。

8 评估

8.1 实验设置

测试平台。 我们分别使用两套配置开展单卡与多卡实验。单卡实验的服务器配有 24 核 AMD EPYC-7V13 CPU 和一张 NVIDIA A100 (80GB)；多卡实验使用 64 核 AMD EPYC CPU 与四张 NVIDIA A6000 (48GB)。两台服务器均运行 CUDA 12.1 和 cuDNN 8.9.2。

负载。 我们测试四类代表性 LLM 应用。每个引擎使用一张 GPU，运行 LLaMA-13B 或 LLaMA-7B [51]。长文档分析使用 Arxiv 数据集 [27]，对学术论文执行链式或 Map-Reduce 摘要。为研究多用户应用的共享机会，我们使用 Bing Copilot 与 GPTs [42] 的提示词，并合成

负载	依赖请求服务	性能目标推导	提示词共享	应用中心调度
数据分析	✓	✓		✓
服务流行 LLM 应用			✓	✓
多智能体应用	✓	✓	✓	✓
混合负载	✓	✓		✓

表 2: 负载及其中生效的优化。

⁵译注：接口字段保持原代码：prompt 为提示词，placeholders 为占位符列表，name 为名称，in_out 标记输入或输出，semantic_var_id 为语义变量标识，transforms 为转换定义，session_id 为会话标识，criteria 为性能指标。

⁶译注：共享的是公共前缀的 KV 数据及其加载机会。不同请求的查询向量仍可能不同，不能把某请求的完整注意力输出无条件当作另一请求的输出；这里的中间统计用于各自查询的分块注意力合并。

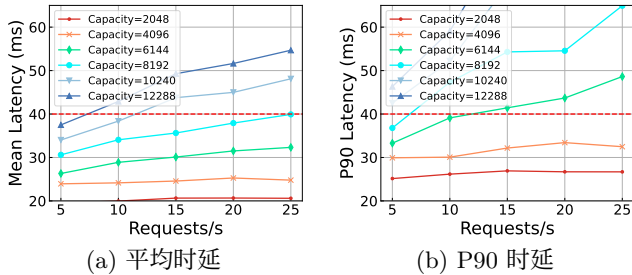


图 10: 不同词元容量与请求率下, vLLM 每输出词元的时延。请求取自 ShareGPT [50], 到达时刻按泊松过程生成。横轴 Requests/s 为每秒请求数, 左图 Mean Latency 为平均时延, 右图 P90 Latency 为时延第 90 百分位数, 单位为毫秒; Capacity 为引擎允许的同时驻留词元容量, 水平虚线为 40 毫秒阈值。

用户查询。多智能体编程应用基于 MetaGPT [22], 包含设计 API 的架构师、分别编写不同文件的程序员, 以及提供意见的审查员; 程序员还会依照意见修改代码。对话服务负载取自反映真实 LLM 对话的 ShareGPT [50]。依据实测分布, 我们为请求引入 200 ~ 300 毫秒随机延迟, 模拟典型互联网网络开销。为使负载接近真实情况, 我们用 GPT-4 [41] 记录回答, 确保性能分析时 LLaMA 生成相近长度的文本。表 2 列出各负载及优化。⁷

基线。 我们与当时构建应用、提供推理服务的先进方案比较。大多数基线应用使用主要开发框架 LangChain [8], 通过 FastChat [62] 提供的 OpenAI 风格对话补全 API 调用服务。FastChat 是广为使用的开源系统, 当时仓库已有超过 30000 个 star。它将到达请求分配给运行 HuggingFace Transformers [53] 或 vLLM [25] 的引擎, 采用 FlashAttention [12]、PagedAttention [25] 和连续批处理 [56] 等优化。FastChat 默认把请求分配到当前队列最短的引擎。由于现有服务通常暴露对话补全 API, 基线把各请求视为独立且高度时延敏感。为控制生成响应时间, 每个引擎设置容量阈值, 即全部活跃请求的总词元数上限。

LLM 词元生成通常受内存带宽限制, 单词元生成时延主要受批次中正在处理的词元数量影响。如图 10 所示, 即使启用连续批处理, vLLM 的每输出词元时间 TPOT 在批次容量超过 6144 后仍明显升高。评测中, 我们采用能把时延敏感请求的生成时延控制在 40 ms/s 以下的配置, 这与使用 OpenAI LLM 服务的经验一致。⁸

当全部引擎达到容量上限后, 额外请求按先入先出 (FIFO) 排队, 等待运行任务完成并释放资源。若要在可接受时延内支持 32k 乃至 1M 词元等更长上下文, 需要更多 GPU 的张量并行 [49]、序列并行 [6], 或

⁷译注: 该评测主要验证系统性能; 使用相近回答长度并不等于验证生成内容或编程结果质量完全相同。网络开销为人工注入, 不能把所有端到端收益解释为纯 GPU 推理加速。

⁸译注: 原文写作 40 ms/s; 根据上下文 TPOT 定义和图中毫秒轴, 应是每输出词元约 40 毫秒的阈值。这里保留原单位并指出歧义, 不能将毫秒每秒当作 TPOT 单位。

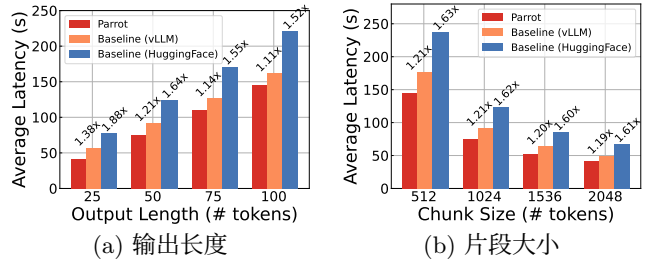


图 11: 不同输出长度和片段大小下, 链式摘要的平均端到端时延。左图 Output Length 为输出词元数, 右图 Chunk Size 为每片词元数, 纵轴 Average Latency 为平均时延 (秒); 柱形对应 Parrot、vLLM 基线与 HuggingFace 基线, 上方数字为相对加速比。

StreamingLLM [55] 等近似注意力方案, 这超出本文范围。

8.2 长文档数据分析

实验从 Arxiv-March 数据集 [27] 随机抽取十篇长文档, 每篇超过 20000 个词元, 执行链式和 Map-Reduce 摘要。结果报告这些文档的平均端到端时延。

链式应用。 我们评估 Parrot 如何降低客户端交互带来的通信开销。图 11 展示一张 A100 上 LLaMA-13B 引擎为单篇文档生成摘要的平均端到端时延。我们调整片段词元数与输出长度, 结果分别展示于图 11a 与图 11b。相对于 vLLM 和 HuggingFace 基线, Parrot 最高分别实现 1.38 $\times$ 与 1.88 $\times$ 加速。主要收益来自减少客户端交互, 从而降低网络时延。输出越长, 生成时间占比越高, 相对收益越小。片段越大, 片段数量越少, 但加速仍取决于每片节省的网络时延。由于词元生成比提示词处理更耗时, 固定输出长度、改变片段大小时, 加速大致保持稳定: 相对于 vLLM 和 HuggingFace 分别为 1.2 $\times$ 和 1.66 $\times$ 。这说明依赖请求优化特别适合摘要、短回答、评分和选择等短输出任务。HuggingFace 比 vLLM 更慢, 后续评测只与 vLLM 比较。⁹

图 12a 加入不同到达率的背景请求, 检验服务如何减轻依赖请求的额外排队。Parrot 相对于 vLLM 基线最高实现 2.38 $\times$ 加速: 首片摘要完成后, 立即将已有摘要加入下一片提示词并继续处理。基线独立处理各请求, 除客户端网络交互外, 后续请求还必须重新进入队列。图 12b 展示多个链式摘要应用并发提交, 每个应用处理不同文档的情况。Parrot 的平均端到端时延获得 1.68 $\times$ 加速, 且如图 13 所示没有让任何一个应用变慢。基线交错执行不同应用, 反而增加了它们的端到端时延。这些实验验证: 理解请求间连接, 比孤立处理请求更能改善整体性能。

⁹译注: 正文称“片段大小与输出长度分别展示于”两子图, 但图名与横轴实际顺序相反; 译文保留引用次序, 具体变量以图注和横轴为准。右图 HuggingFace 加速标注约为 1.60~1.63 倍, 正文概括为 1.66 倍, 二者略有差异。

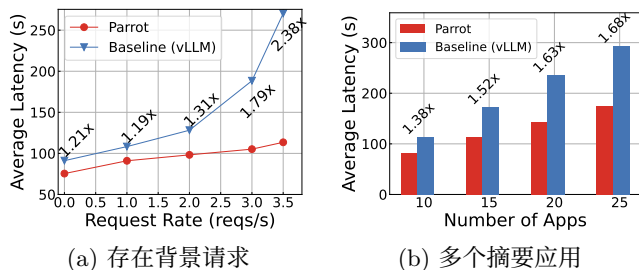


图 12: 存在背景请求或其他链式摘要应用时, 链式摘要的平均端到端时延。左图 Request Rate 为背景请求率 (请求/秒), 右图 Number of Apps 为应用数量, 纵轴 Average Latency 为平均时延 (秒); 比较 Parrot 与 vLLM 基线。

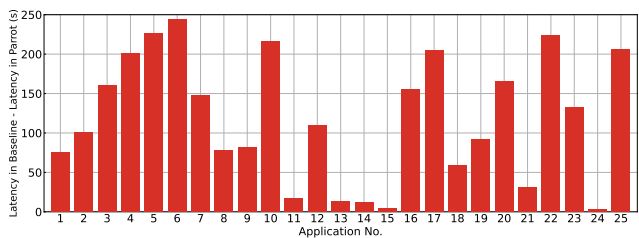


图 13: 25 个链式摘要应用在基线与 Parrot 中的端到端时延差; 所有应用均在 Parrot 中更早完成。横轴 Application No. 为应用编号, 纵轴为基线时延减去 Parrot 时延 (秒), 正值表示 Parrot 更快。

Map-Reduce 应用。 文档摘要也可以使用图 1a 的 Map-Reduce 实现: 多个并行请求分别总结不同片段, 再由一个 Reduce 请求将它们合成最终摘要。图 14 显示, 单个 A100、LLaMA-13B 引擎上, Parrot 相对于基线实现 2.37 $\times$ 加速。由于 Map 请求独立, 两系统都会并发提交; Parrot 的主要优势是通过目标推导识别 Map 任务组, 采用更大批次提高吞吐量, 缩短整组完成时间。基线孤立处理请求, 假设它们都对时延敏感, 因而将容量限制为 4096 个词元, 以改善单任务时延, 却损害应用整体性能。这强调服务必须区分请求特征, 才能优化不同应用的端到端性能。

8.3 服务流行的 LLM 应用

生产应用需要服务大量用户。如图 5 所示, 开发者常用很长的系统提示词定义模型行为, 同一应用用户因此共享提示词, 可受益于上下文分叉及前缀共享请求共置调度。由于无法访问 Bing Copilot 的中间步骤, 我们只评估最终生成用户回答的请求; 依据测得的长度分布合成 64 个请求, 系统提示词约 6000 个词元, 输出为 180 至 800 个词元。图 15 展示平均请求时延。基线不知道提示词结构, 难从海量请求识别共性; 相对于不共享前缀的基线, Parrot 在批量大小 8 和 16 时获得 1.8 $\times$ ~ 2.4 $\times$ 加速。继续增大批次会因公共提示词的巨大缓存而显存不足。我们还实现使用 vLLM PagedAttention、共享静

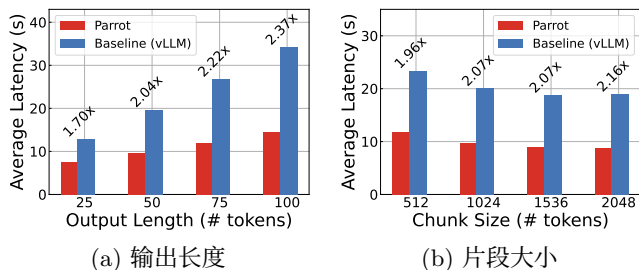


图 14: 不同输出长度与片段大小下, Map-Reduce 文档摘要的平均端到端时延。左图 Output Length 为输出词元数, 右图 Chunk Size 为每片词元数, 纵轴 Average Latency 为平均时延 (秒); 比较 Parrot 与 vLLM 基线, 柱形上方为相对加速比。

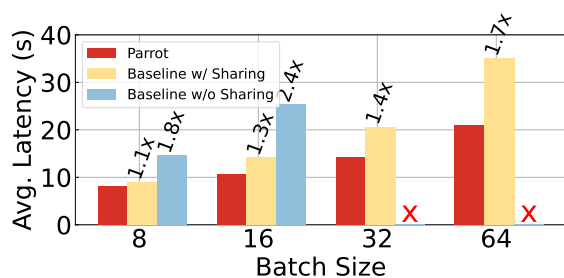


图 15: 不同批量大小下, Bing Copilot 请求的时延。横轴 Batch Size 为批量大小, 纵轴 Avg. Latency 为平均时延 (秒); Baseline w/ Sharing 为共享前缀基线, Baseline w/o Sharing 为不共享基线。红色叉号表示显存不足。

态前缀的增强基线。两者均用分页内存管理 [25], 因此在 A100、LLaMA-7B 引擎中可容纳相同词元数; Parrot 仍因更好的内核获得 1.1 $\times$ ~ 1.7 $\times$ 加速。vLLM 虽减少公共提示词重复占用, 其内核仍反复加载共享词元; 词元生成受内存带宽限制, 这些重复加载拖慢推理。结合 FlashAttention 与 PagedAttention 后, Parrot 为不同用户的新词元计算注意力时, 只需加载公共前缀的 KV 数据一次。收益主要来自解码, 输出越长, 加速越明显。图 16 中, 相对于使用 PagedAttention 的 vLLM, Parrot 分别获得 1.58 $\times$ 和 1.84 $\times$ 加速; 批量大小 32 时, 每输出词元时延约 40 ms。¹⁰

图 17 进一步在多卡集群上评估多个 GPTs 应用 [42], 每个应用有多个用户。四张 A6000 (48GB) 各部署一个 LLaMA-7B 引擎。我们选择生产力、编程、图像生成、数据分析四类流行 GPTs 应用, 以等概率随机生成各类请求, 并按给定平均速率的泊松过程生成到达时刻。相对于不共享前缀的基线, Parrot 可持续处理的请求率提高至 12 $\times$ 。基线不知道应用内部提示词共享, 将请求混合分散到所有引擎, 无法复用公共前缀; Parrot 把同类应用请求共置, 增大共享机会, 兼顾更低推理时延与更高集群吞吐量。关闭这一亲和调度后, 相对基线仅剩

¹⁰ 译注: 这里测试的是由 Bing Copilot 长度分布合成的最终请求, 不能视为测量了 Bing Copilot 完整生产 workflow。两个最高加速比分别来自批量大小 32 和 64 的子图, 后者使用的输出长度范围以图轴为准。

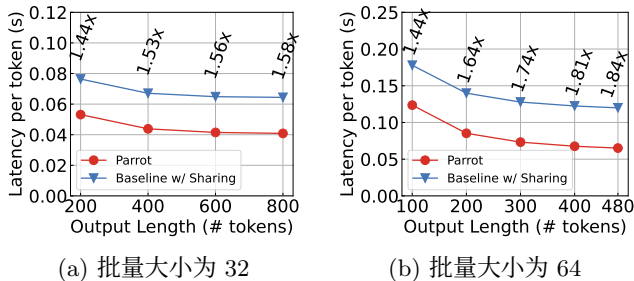


图 16: Bing Copilot 每输出词元时延。横轴 Output Length 为输出词元数, 纵轴 Latency per token 为每词元时延 (秒); 比较 Parrot 与共享前缀基线, 数字为相对加速比。

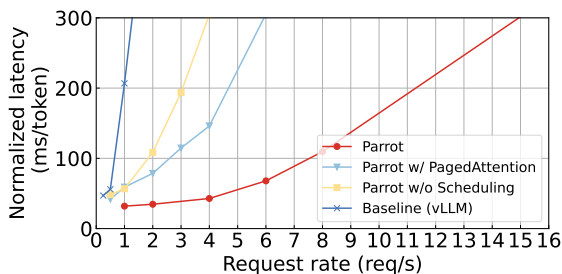


图 17: 服务多个 GPTs 应用。横轴 Request rate 为请求率 (请求/秒), 纵轴 Normalized latency 为按输出长度归一化的请求时延 (毫秒/词元); Parrot w/ PagedAttention 为使用 PagedAttention 内核的 Parrot, Parrot w/o Scheduling 为关闭亲和调度的 Parrot, Baseline (vLLM) 为 vLLM 基线。

3 $\times$ 请求率, 因为公共前缀请求经常落到不同引擎, 共享机会减少。此外, 相较于使用 vLLM PagedAttention 的 Parrot, 其专用注意力内核通过避免共享前缀的重复数据加载, 使可支持请求率进一步提高至 2.4 $\times$ 。¹¹

8.4 多智能体应用

我们在 Parrot 中使用 MetaGPT [22] 评估多智能体系统。工作流包含三类角色: 架构师首先为指定任务设计项目文件结构及每个文件的 API; 随后多个程序员分别实现各文件; 代码合并后, 多个审查员分别检查一个文件并提出意见; 程序员依据意见修改代码。审查与修改共迭代三轮, 生成最终代码。图 18 比较单张 A100、LLaMA-13B 上的时延与显存占用。相对于时延优先基线, Parrot 最高获得 11.7 $\times$ 加速, 主要收益来自依据端到端目标推导请求性能目标。本场景以尽快交付最终代码为目标; 系统识别并行编程、审查、修改过程中的多个任务组, 使用更大批次, 提高吞吐量并缩短整组完成时间。我们还比较有采用大批次、优化集群吞吐量的

¹¹ 译注: 本段评测使用合成用户查询及四类提示词, 测量的是文本 LLM 服务性能, 不是完整图像生成或工具调用链的执行性能。12 倍属于该模型、前缀复用度、调度与集群配置下的最大可持续请求率比较。

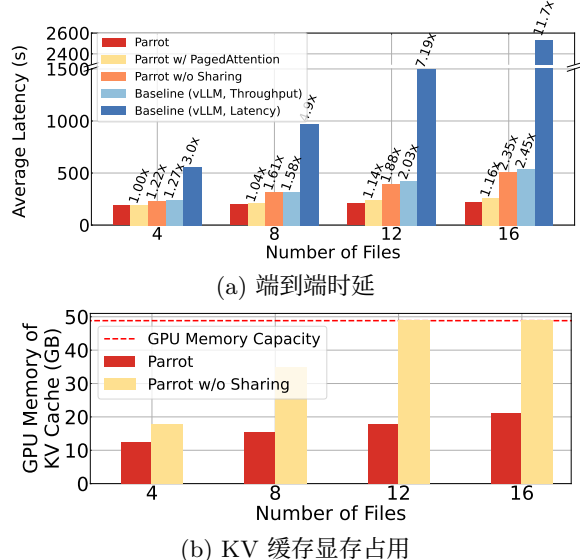


图 18: 不同待编写文件数量下, 多智能体编程的时延与显存占用。横轴 Number of Files 为文件数, 上图 Average Latency 为平均时延 (秒); 比较 Parrot、使用 PagedAttention 的 Parrot、关闭共享的 Parrot, 以及吞吐量优先/时延优先 vLLM 基线。下图 GPU Memory of KV Cache 为缓存占用 (GB), GPU Memory Capacity 虚线为缓存可用容量; 关闭共享的系统到达该上限。

基线; 它相较于时延优先基线, 也具有更高并发和更短完成时间。¹²

即使与吞吐量优先基线比较, Parrot 仍最高获得 2.45 $\times$ 加速。这一改善主要来自提示词结构分析减少重复内容, 对应 2.35 $\times$ 加速。MetaGPT 的不同角色相互交互, 具有大量重叠上下文, Parrot 将其作为公共前缀共享。vLLM 的静态前缀共享机制在此动态场景中不起作用; 不了解提示词结构, 就无法识别运行时新生成、仍可共享的语义变量。如图 18b 所示, 关闭共享后, Parrot 会达到显存上限。另外, 在编写 16 个文件时, 其共享前缀专用内核相对于使用 vLLM PagedAttention, 再带来 1.2 $\times$ 加速, 原因是减少内存事务。

8.5 混合负载调度

为评估多卡性能, 我们使用四张 A6000 (48GB), 各运行一个独立 LLaMA-7B 引擎, 共四个引擎。模拟公共服务面对多样化需求的场景, 混合注入到达率为每秒 1 个请求的对话负载, 以及 §8.2 分析的 Map-Reduce 数据分析负载。前者需要低时延, 后者优先追求高吞吐量, 共用同一引擎时发生冲突。我们比较两个基线: 时延优先基线限制引擎容量, 减少解码时间; 吞吐量优先基线使用全部容量, 最大化 GPU 利用率。

¹² 译注: 这里的三轮审查修改是已设定轮数的评测工作流, 不代表本文已实现讨论部分留待未来的一般动态循环或概率分支。加速结果属于系统完成时间指标, 未给出代码正确率的对应提升。

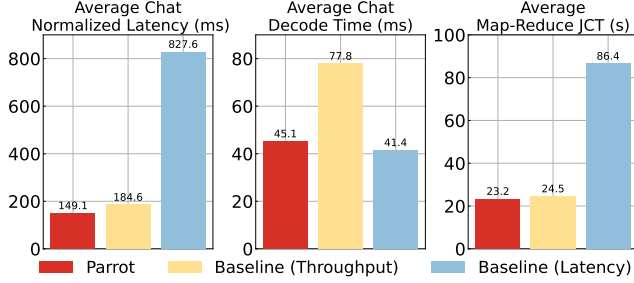


图 19: 对话与 Map-Reduce 应用的混合负载。左图 Average Chat Normalized Latency 为平均对话归一化时延 (原图标毫秒, 按正文定义为毫秒/输出词元), 中图 Average Chat Decode Time 为平均对话解码时间 (毫秒), 右图 Average Map-Reduce JCT 为 Map-Reduce 平均作业完成时间 (秒); 比较 Parrot、吞吐量优先基线和时延优先基线。

图 19 显示, 与时延优先和吞吐量优先基线相比, Parrot 的对话归一化时延, 即请求时延除以输出词元数 [25, 56], 分别改善 $5.5\times$ 和 $1.23\times$ 。对话词元生成速度与时延优先基线接近, 比吞吐量优先基线快 $1.72\times$ 。Map-Reduce 应用则相对于时延优先基线获得 $3.7\times$ 加速, 比吞吐量优先基线快 $1.05\times$ 。Parrot 将两类负载合理调度到不同引擎, 减轻争用, 同时提供对话低时延与分析高吞吐量。这说明针对多样化请求进行区别处理, 是提高整体性能的重要条件。

9 相关工作

深度学习服务系统。 近年来模型服务研究活跃, 许多系统针对部署中的不同挑战提出方案, 例如 Clipper [10]、TensorFlow Serving [39]、Clockwork [19]、REEF [20] 和 AlpaServe [28], 涵盖单模型或多模型的批处理、缓存、放置、调度及模型并行。这些系统面向通用深度学习模型, 对 LLM 的自回归解码等独特需求考虑较少。Orca [56] 提出迭代级细粒度调度, 将多个请求连续合批; vLLM 的 PagedAttention [25] 使用非连续内存, 对不同长度请求合批, 提高显存利用率。然而, 这些系统仍独立处理请求, 未利用应用内部连接和请求间共性。Parrot 与它们的方法相互独立: 通过语义变量暴露更多应用信息, 再分析请求数据流, 打开新优化空间, 最终追求应用端到端性能。

LLM 编排框架。 这些框架帮助开发者创建和管理 LLM 应用, 简化提示词设计与多请求编排。LangChain [8] 是 Python 框架, 提供链式、Map-Reduce 等工作流模式, 便于定制应用。Semantic Kernel [36] 的 Planner 是可依据用户需求自动制定计划的语义智能体。PromptFlow [35] 支持原生函数和语义函数链, 并可视化。LlamaIndex [29] 支持自然语言查询检索相关文档。Parrot 与这些框架相互独立, 可方便地集成, 使其支持带语义变量抽象的 API, 详见 §6。

利用 DAG 的系统优化。 依赖图或有向无环图广泛存在于各类系统, 许多工作利用 DAG 信息优化性能。Tez [4]、Dryad [23] 和 Graphene [16] 利用任务依赖改善并行数据分析的调度与资源装箱。SONIC [30]、Caerus [59] 和 Orion [31] 从通信、时延与成本等方面优化无服务器函数。Parrot 借鉴既有系统工作, 认识到请求关联对于端到端性能的重要性, 因此设计 API 暴露依赖信息。另外, 除请求依赖外, LLM 应用还特有地需要理解提示词结构, 这对通信和识别共性都很重要。因此, 我们提出语义变量抽象, 而不仅仅使用请求 DAG。

10 结论

本文提出 Parrot, 将 LLM 应用视为一等对象, 优化应用端到端性能。我们提出关键的语义变量抽象, 暴露请求依赖与共性, 创造新的优化空间。评测表明, Parrot 对 LLM 应用最高实现 $11.7\times$ 加速。我们认为, 从这一新角度提高效率, 还能带来广阔的研究方向, 例如重新研究应用端到端性能的公平性等调度功能。

致谢

我们感谢匿名审稿人和指导审稿人提出的建设性意见与建议。Zhenhua Han、Yuqing Yang 和 Chen Chen 为通讯作者。

参考文献

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for Large-Scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, Savannah, GA, November 2016. USENIX Association.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *arXiv preprint arXiv:2403.02310*, 2024.
- [3] Ganesh Ananthanarayanan, Srikanth Kandula, Albert Greenberg, Ion Stoica, Yi Lu, Bikas Saha, and Edward Harris. Reining in the outliers in Map-Reduce clusters using mantri. In *9th USENIX*

Symposium on Operating Systems Design and Implementation (OSDI 10), Vancouver, BC, October 2010. USENIX Association.

- [4] Apache. Tez. <https://tez.apache.org/>, November 2019.
- [5] Apache. Kafka. <https://kafka.apache.org/>, October 2023.
- [6] Zhengda Bian, Hongxin Liu, Boxiang Wang, Haichen Huang, Yongbin Li, Chuanrui Wang, Fan Cui, and Yang You. Colossal-ai: A unified deep learning system for large-scale parallel training. *CoRR*, abs/2110.14883, 2021.
- [7] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4, 2023.
- [8] Harrison Chase. LangChain. <https://github.com/langchain-ai/langchain>, October 2022.
- [9] Lequn Chen. Dissecting batching effects in gpt inference. <https://le.qun.ch/en/blog/2023/05/13/transformer-batching/>, May 2023.
- [10] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A Low-Latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, Boston, MA, March 2017. USENIX Association.
- [11] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [12] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022.
- [13] Bill Gates. Ai is about to completely change how you use computers and upend the software industry. <https://www.gatesnotes.com/AI-agents>, Nov 2023.
- [14] Google. Google bard. <https://bard.google.com/>, Nov 2023.
- [15] Robert Grandl, Mosharaf Chowdhury, Aditya Akella, and Ganesh Ananthanarayanan. Altruistic scheduling in Multi-Resource clusters. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 65–80, Savannah, GA, November 2016. USENIX Association.
- [16] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. GRAPHENE: Packing and Dependency-Aware scheduling for Data-Parallel clusters. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 81–97, Savannah, GA, November 2016. USENIX Association.
- [17] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, Boston, MA, February 2019. USENIX Association.
- [18] guidance ai. Guidance. <https://github.com/guidance-ai/guidance>, November 2023.
- [19] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, November 2020.
- [20] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, Carlsbad, CA, July 2022. USENIX Association.
- [21] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. DeepSpeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671*, 2024.
- [22] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran,

- et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [23] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed data-parallel programs from sequential building blocks. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems 2007, EuroSys '07*, page 59–72, New York, NY, USA, 2007. Association for Computing Machinery.
- [24] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R. Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 642–657, New York, NY, USA, 2023. Association for Computing Machinery.
- [25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.
- [26] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>, 2022.
- [27] Yucheng Li. Unlocking context constraints of llms: Enhancing context efficiency of llms with self-information-based content filtering, 2023.
- [28] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, Boston, MA, July 2023. USENIX Association.
- [29] Jerry Liu. LlamaIndex, November 2022.
- [30] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. SONIC: Application-aware data passing for chained serverless applications. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 285–301. USENIX Association, July 2021.
- [31] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 303–320, Carlsbad, CA, July 2022. USENIX Association.
- [32] Microsoft. Bing chat. <https://www.bing.com/chat>, Nov 2023.
- [33] Microsoft. Meeting recap in microsoft teams. <https://www.microsoft.com/en-us/microsoft-teams/premium>, May 2023.
- [34] Microsoft. Microsoft 365 copilot. <https://www.microsoft.com/en-us/microsoft-365/enterprise/microsoft-365-copilot>, Mar 2023.
- [35] Microsoft. PromptFlow. <https://github.com/microsoft/promptflow>, November 2023.
- [36] Microsoft. Semantic Kernel. <https://github.com/microsoft/semantic-kernel>, November 2023.
- [37] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, November 2020.
- [38] Flemming Nielson, Hanne R Nielson, and Chris Hankin. *Principles of program analysis*. Springer, 2015.
- [39] Christopher Olston, Fangwei Li, Jeremiah Hansen, Jordan Soyke, Kiril Gorovoy, Li Lao, Noah Fiedel, Sukriti Ramesh, and Vinu Rajashekhar. Tensorflow-serving: Flexible, high-performance ml serving. In *Workshop on ML Systems at NIPS 2017*, 2017.
- [40] OpenAI. Chatgpt. <https://chat.openai.com/>, Nov 2023.
- [41] OpenAI. Gpt-4 technical report, 2023.
- [42] OpenAI. Introducing gpts. <https://openai.com/blog/introducing-gpts>, Nov 2023.

- [43] OpenAI. OpenAI Triton. <https://github.com/openai/triton>, November 2023.
- [44] OpenAI. Production best practices - openai api. <https://platform.openai.com/docs/guides/production-best-practices/improving-latencies>, Nov 2023.
- [45] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [46] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. *arXiv preprint arXiv:2311.18677*, 2023.
- [47] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- [48] Sebastián Ramírez. FastAPI. <https://github.com/tiangolo/fastapi>.
- [49] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.
- [50] ShareGPT Team. Sharegpt dataset. <https://sharegpt.com/>, Nov 2023.
- [51] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [52] Unknown. Prompt of bing chat. https://www.make-safe-ai.com/is-bing-chat-safe/Prompts_Conversations.txt, Nov 2023.
- [53] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-Art Natural Language Processing. pages 38–45. Association for Computational Linguistics, October 2020.
- [54] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Auto-gen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- [55] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv*, 2023.
- [56] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.
- [57] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems*, EuroSys '10, page 265–278, New York, NY, USA, 2010. Association for Computing Machinery.
- [58] Matei Zaharia, Andy Konwinski, Anthony D Joseph, Randy H Katz, and Ion Stoica. Improving mapreduce performance in heterogeneous environments. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)*, San Diego, CA, 2008.
- [59] Hong Zhang, Yupeng Tang, Anurag Khandelwal, Jingrong Chen, and Ion Stoica. Caerus: NIMBLE task scheduling for serverless analytics. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 653–669. USENIX Association, April 2021.
- [60] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.
- [61] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis C.M. Lau, Yuqi Wang, Yifan Xiong, and Bin Wang. HiveD: Sharing a GPU cluster for deep learning with guarantees. In *14th USENIX Symposium on Operating Systems Design and Implementation*

(*OSDI 20*), pages 515–532. USENIX Association, November 2020.

- [62] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.
- [63] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Efficiently programming large language models using sglang, 2023.
- [64] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.